

CASA DO CÓDIGO — EDIÇÃO REVISADA

Ruby on Rails

Coloque sua aplicação web nos trilhos com as melhores práticas

★ Edição Definitiva: Atualizada para Rails 8 & Ruby 3.4 ★

Autor: Vinícius B. Fuentes

GUIA PRÁTICO E INTEGRAL COM O PROJETO COLCHO.NET

Ruby on Rails: Coloque sua aplicação web nos trilhos

Edição Integral e Definitiva: Atualizada para Rails 8 e Ruby 3.4

Baseado na obra original de Vinícius B. Fuentes (Casa do Código)

Texto integral, didático e sem cortes, com todas as seções, caixas de dicas e notas conceituais adaptadas para o ecossistema Rails 8.

Sumário

- Agradecimentos
- Capítulo 1: Introdução
 - 1.1 Nova edição, atualizado para Rails 8!
 - 1.2 Para quem é este livro
 - 1.3 Organização da obra
 - 1.4 Socorro, estou perdido! Me ajude!
- Capítulo 2: Conhecendo a aplicação
 - 2.1 Arquitetura de aplicações web
 - 2.2 Recursos ao invés de páginas
 - 2.3 Recursos no Colcho.net
 - 2.4 Conhecendo os componentes
 - 2.5 Os modelos
 - 2.6 Controle
 - 2.7 Apresentação
 - 2.8 Rotas
 - 2.9 Suporte
 - 2.10 Considerações finais
- Capítulo 3: Primeiros passos com Rails

- 3.1 Instalação do Ruby e Rails
- 3.2 Gerar o alicerce da aplicação
- 3.3 Os ambientes de execução
- 3.4 Os primeiros comandos
- 3.5 Os arquivos gerados pelo scaffold
- Capítulo 4: Implementação do modelo para o cadastro de usuários
- 4.1 O usuário
- 4.2 Evite dados errados. Faça validações
- Capítulo 5: Tratando as requisições Web
- 5.1 Roteie as requisições para o controle
- 5.2 Integre o controle e a apresentação
- 5.3 Controle o mass-assignment com Strong Parameters
- 5.4 Exibição do perfil do usuário
- 5.5 Permita a edição do perfil
- 5.6 Reaproveite as apresentações com partials
- 5.7 Mostre os erros no formulário
- 5.8 Configure a ação raiz (root)
- Capítulo 6: Melhore o projeto
- 6.1 Lição obrigatória: sempre aplique criptografia para armazenar senhas
- 6.2 Como adicionar plugins e dependências ao projeto?
- 6.3 Usando `has_secure_password` no modelo
- 6.4 Migração da tabela users
- 6.5 Automatizando tarefas de manutenção com Rake
- 6.6 Melhoria de templates e CSS
- 6.7 Trabalhe com layout e templates para melhorar sua apresentação
- 6.8 O que é o Asset Pipeline contemporâneo (Propshaft e Importmaps)?
- 6.9 Criando os novos stylesheets
- 6.10 Feedback em erros de formulário
- 6.11 Duplicação de lógica na apresentação nunca mais. Use os Helpers
- Capítulo 7: Faça sua aplicação falar várias línguas
- 7.1 O processo de internacionalização (I18n)

- 7.2 Traduza os templates
- 7.3 Extra: alterar o idioma do site
- Capítulo 8: O cadastro do usuário e a confirmação da identidade
- 8.1 Entenda o ActionMailer e use o MailCatcher / Letter Opener
- 8.2 Templates de email, eu preciso deles?
- 8.3 Mais emails e a confirmação da conta de usuário
- 8.4 Um pouco de callbacks para realizar tarefas pontuais
- 8.5 Roteamento com restrições e controllers singleton
- 8.6 Em resumo
- Capítulo 9: Login do usuário
- 9.1 Trabalhe com a sessão
- 9.2 Controles e rotas para o novo recurso
- 9.3 Sessões e cookies
- 9.4 Consultas no banco de dados
- 9.5 Escopo de usuário confirmado
- Capítulo 10: Controle de acesso
- 10.1 Helpers de sessão
- 10.2 Não permita edição do perfil alheio
- 10.3 Relacionando seus objetos
- 10.4 Relacione quartos a usuários
- 10.5 Limite o acesso usando relacionamentos
- 10.6 Exibição e listagem de quartos
- Capítulo 11: Avaliação de quartos, relacionamentos muitos para muitos e organização do código
- 11.1 Relacionamentos muitos-para-muitos
- 11.2 Removendo objetos sem deixar rastros
- 11.3 Criando avaliações reativas com Hotwire / Turbo Streams
- 11.4 Diga adeus a regras complexas de apresentação: use presenters
- 11.5 Da era jQuery ao Hotwire: requisições assíncronas modernas
- 11.6 Média de avaliações usando agregações
- 11.7 Aplicações modernas usam tipografia e ícones modernos
- 11.8 Eu vejo estrelas: usando CSS e Stimulus para melhorar as avaliações

- 11.9 Encerrando
 - Capítulo 12: Polindo o Colcho.net
 - 12.1 Faça buscas textuais apenas com o Rails
 - 12.2 URLs mais amigáveis através de slugs
 - 12.3 Paginação de dados de forma descomplicada
 - 12.4 Upload de fotos de forma simples com Active Storage
 - 12.5 Coloque a aplicação no ar
 - Capítulo 13: Próximos passos
-

Agradecimentos

**Monsters are real, and ghosts are real too. They live inside us, and sometimes, they win.* – Stephen King*

Este livro não existiria sem a dedicação de grandes amigos da comunidade Ruby e da equipe da Casa do Código e Caelum, que revisaram, criticaram e opinaram minuciosamente sobre o conteúdo deste material. Agradecimentos especiais ao GURU-SP (Grupo de Usuários Ruby de São Paulo), à antiga PlataformaTec e a todos que colaboraram ao longo dos anos para disseminar o Ruby no Brasil.

Por fim, agradeço principalmente a você, leitor, por investir seu tempo em aprender uma tecnologia tão prazerosa e produtiva quanto o Ruby on Rails. Esperamos que esta jornada com o Rails 8 seja divertida e lucrativa ao mesmo tempo!

Capítulo 1: Introdução

**Quem, de três milênios, / Não é capaz de se dar conta / Vive na ignorância, na sombra, / À mercê dos dias, do tempo.* – Goethe*

Se você desenvolve para a Web, provavelmente já ouviu falar sobre Ruby on Rails. O Ruby on Rails (ou também apenas "Rails") é um framework open-source para desenvolvimento de aplicações web, criado por David Heinemeier Hansson (ou "DHH"). O framework, escrito na linguagem Ruby, foi extraído de um produto de sua empresa, o Basecamp® (<http://basecamp.com>), em 2003.

Desde então ele ficou muito famoso, levando também a linguagem Ruby, anteriormente conhecida apenas no Japão e em poucos lugares dos Estados Unidos, ao mundo todo. Mas por que cresceu tanto? O que a linguagem e o framework trouxeram de novo para sair do anonimato e praticamente dominar o mercado de startups nos Estados Unidos e no mundo?

Quanto ao Rails, na época em que foi lançado, trouxe uma visão completamente diferente ao desenvolvimento Web. Naquele momento, desenvolver para Web era cansativo, os frameworks eram complicados ou resultavam em sistemas difíceis de se manter e de baixa qualidade.

O DHH, ao desenvolver o Basecamp, pensou principalmente nos seguintes aspectos:

- **"Convention over configuration" (Convenção à configuração):** ao invés de configurar um conjunto sem fim de arquivos XML ou JSON, adota-se a convenção de nomes e estruturas, e apenas muda-se o que for estritamente necessário;
- **"Don't Repeat Yourself" (DRY - Não se repita):** nunca você deve fazer mais de uma vez o que for necessário (como duplicar uma regra de negócio ou mapeamento de dados);
- **Automação de tarefas repetidas:** nenhum programador deve perder tempo em tarefas repetitivas e burocráticas, e sim investir seu tempo em resolver problemas interessantes e gerar valor de negócio.

Esses conceitos são amplamente explorados no clássico *The Pragmatic Programmer: From Journeyman to Master*, leitura altamente recomendada. Esta soma de tecnologias e práticas torna o trabalho diário extremamente prazeroso; é possível realizar muito com pouco tempo e você verá, capítulo a capítulo, como essas ideias são representadas em todos os aspectos do framework.

1.1 Nova edição, atualizado para Rails 8!

Na edição original deste livro, cobríamos a transição para o Rails 4. O tempo passou, e o Rails continuou inovando radicalmente. Nesta edição atualizada para o **Rails 8 e Ruby 3.4**, vamos assumir que você já possui noções básicas de programação em Ruby e nos dedicaremos integralmente às convenções contemporâneas do ecossistema:

- O abandono de dependências legadas de JavaScript (como jQuery e compiladores externos pesados) em favor de **Hotwire (Turbo + Stimulus)** e **Importmaps**;
- A substituição de soluções antigas de upload de arquivos (CarrierWave) pelo **Active Storage** nativo;
- O gerenciamento moderno de tarefas em background e cache com o **Solid Suite** do Rails 8;
- As novas macros de autenticação, tokens seguros e o modelo de **Strong Parameters** consolidado.

1.2 Para quem é este livro

O objetivo deste livro é apresentar o ecossistema e ensinar os passos fundamentais para desenvolver aplicações profissionais com o framework Ruby on Rails. Mais além, vamos aprender a desenvolver aplicativos organizados, elegantes e robustos. Algumas vezes usaremos inclusive componentes feitos em Ruby puro (como *Presenters* e modelos de formulário com *ActiveModel*). Esses momentos são muito importantes para aprendermos também excelentes práticas de arquitetura.

Tendo isso em mente, este livro serve para pessoas que:

- Estão começando agora no desenvolvimento web com Ruby on Rails;
- Já conhecem versões legadas do Rails (como Rails 3, 4 ou 5) e querem se atualizar com a arquitetura contemporânea do Rails 8;
- Já conhecem Rails superficialmente, mas não estão confortáveis em como fazer aplicações bem organizadas e desacopladas;
- Querem aprimorar seus conhecimentos de boas práticas no design de software web.

1.3 Organização da obra

A primeira parte do livro é dedicada a entender o contexto em que o Ruby on Rails trabalha. Essa é a parte teórica, na qual vamos entender quais são os principais conceitos por trás do framework. Vamos ver também, em alto nível, quais são os componentes do Rails e como eles se relacionam.

A segunda parte é onde vamos fazer a aplicação prática. Passo a passo, vamos implementar uma aplicação do início ao fim e, durante a construção de cada funcionalidade, aprender como juntar as partes do framework. Vamos revisar funcionalidades, aprimorando-as, e mostrar como é o

processo de criação de uma aplicação real, de forma que você aprenda uma das possíveis maneiras de construir suas próprias aplicações no futuro.

Vamos construir um aplicativo chamado **Colcho.net**. O Colcho.net é um site para você publicar um espaço sobrando na sua casa para hospedar alguém por uma ou mais noites (um colchonete na sala ou um quarto aconchegante). O site vai ter:

- Cadastro de usuário, com validações robustas e encriptação segura de senha;
- Login e gerenciamento de sessões seguras com cookies assinados;
- Envio de e-mails de boas-vindas e confirmação de conta com tokens únicos;
- Internacionalização (I18n) completa em português e inglês com slugs na URL;
- Publicação, listagem e administração restrita de quartos;
- Upload nativo de fotos e geração de miniaturas com Active Storage;
- Avaliação de quartos com notas de 1 a 5 estrelas e interface reativa via Stimulus;
- Busca textual com destaque de termos (highlight) e paginação eficiente de resultados;
- Preparação para publicação e deploy em nuvem.

Embora este livro tenha sido construído de forma que a leitura progressiva seja natural, ele também serve como um excelente manual de consulta para o seu dia a dia.

1.4 Socorro, estou perdido! Me ajude!

A comunidade Ruby on Rails é reconhecida mundialmente por seu acolhimento e riqueza de materiais. Sempre que encontrar dúvidas ou comportamentos inesperados, recorra às referências oficiais atualizadas:

- Guias oficiais: [RailsGuides](#)
 - Documentação da API: [Rails API](#)
 - Comunidades ativas no Brasil (como o GURU-SP e fóruns de Ruby Brasil).
-

Capítulo 2: Conhecendo a aplicação

Para cada fato existe um conjunto infinito de hipóteses. Quanto mais você observa, mais você enxerga.
— Robert Pirsig, em **Zen and the Art of Motorcycle Maintenance**

Antes de começarmos, você sabe qual a diferença entre um framework e uma biblioteca? Quando usamos uma biblioteca, nós programadores escrevemos nosso próprio código e chamamos as funções da biblioteca quando necessário. A camada entre a biblioteca e o nosso código é bastante distinta e nós controlamos o fluxo da aplicação.

O mesmo não acontece quando usamos um framework. Para se ter uma ideia, muitas vezes o ponto inicial do sistema não é um código que você escreve: o framework assume o controle (*Inversão de Controle*), e nós plugamos o nosso código nos pontos adequados que o framework expõe. Nosso código faz o intermédio com diversas outras ferramentas que compõem a estrutura, de forma que o resultado torna-se muito mais poderoso do que a simples soma das partes isoladas.

O Ruby on Rails adota o padrão de arquitetura chamado **Model-View-Controller (MVC)**, ou Modelo, Apresentação e Controle, bastante conveniente para a construção de aplicativos Web. O Rails também é usado por gigantes da internet há mais de duas décadas (GitHub, Shopify, Airbnb, Basecamp, etc.), portanto já foi exaustivamente testado em cenários de tráfego massivo e alta carga. É fácil começar com Rails porque ele faz muito trabalho por você; mas aprofundar-se no Rails te dá ainda mais poder.

2.1 Arquitetura de aplicações web

Construir aplicações bem-feitas em Ruby on Rails é um pouco mais estruturado do que simplesmente criar scripts que geram páginas atrás de páginas. A razão disso é que ele é preparado para criar aplicações modernas, modulares e arrojadas. Isso significa que não somente deve responder HTML aos usuários, mas também responder de maneira adequada para requisições parciais (Turbo Frames/Streams) ou fornecer representações estruturadas (como JSON para APIs).

NOTA

****O que são aplicações ricas em client-side e a abordagem do Hotwire?**** Historicamente, para criar telas reativas sem recarregar o navegador por inteiro, a indústria recorria a grandes frameworks JavaScript no navegador (como React, Vue ou os antigos Angular e Backbone), tratando o back-end apenas como uma API JSON. O Rails moderno revolucionou esse paradigma com o ****Hotwire (HTML Over The Wire)****. Em vez de duplicar a lógica de templates no navegador e no servidor, o Rails envia fragmentos de HTML pré-renderizados diretamente do servidor e substitui partes cirúrgicas da tela em milissegundos. Assim, você ganha a velocidade e a fluidez de uma Single Page Application sem abrir mão da simplicidade e da produtividade do back-end em Ruby.

2.2 Recursos ao invés de páginas

Para que nossas aplicações fiquem elegantes, precisamos parar de pensar na maneira antiquada de se criar aplicações Web cheias de "páginas soltas" com interações complexas entre si, e pensarmos em **recursos** (*resources*). É uma mudança de mentalidade não trivial para quem vem de abordagens procedurais, mas vamos exercitar esse pensamento durante o livro inteiro, e logo ele se tornará uma segunda natureza para você.

Se você já ouviu falar ou sabe o que é **REST (Representational State Transfer)**, entender como vamos usar recursos para modelar nossa aplicação será muito natural.

2.3 Recursos no Colcho.net

Um recurso, em uma aplicação web, é qualquer entidade de negócio que uma aplicação pode expor e manipular. Por exemplo, em uma rede social, recursos podem ser mensagens, fotos, vídeos. Uma aplicação web, quando serve um recurso, permite que o usuário interaja com ele: criar novos registros, listá-los, visualizar seus detalhes, editá-los ou removê-los. Um recurso também possui representações (por exemplo, um documento HTML estilizado ou uma carga de dados em JSON).

O Colcho.net é um aplicativo em que os seus usuários podem cadastrar quartos vagos ou um colchonete que pode ser alugado em sua sala.

Assim, o primeiro recurso que podemos identificar é o **Quarto** (`Room`). Em nosso sistema, será possível listar quartos, exibir detalhes de um quarto, criar um novo quarto, editar um quarto já existente e, por fim, removê-lo.

Recursos assim são fáceis de serem mapeados, pois estão diretamente ligados a uma unidade de persistência clara no sistema. Porém, existem recursos que são menos óbvios. Um exemplo de recurso não tão óbvio é a **Sessão** (`Session`). Poderemos criar uma sessão (ao efetuar login) ou destruí-la (ao efetuar logout), porém nada mais além disso. Não há "listagem de sessões" para o usuário comum.

Em sistemas REST, ações com recursos são mapeadas em duas partes fundamentais:

12. Um **verbo HTTP** (que define a operação a ser realizada);
13. Uma **URL** (que identifica a rota ou endereço do recurso).

NOTA

****O que são verbos HTTP?**** Verbos HTTP são ações semânticas padronizadas executadas sobre um recurso identificado por uma URL. Os verbos fundamentais usados no Ruby on Rails são: - ****GET****: recupera a representação de um recurso específico ou de uma coleção; - ****POST****: submete dados para criar um novo elemento na coleção; - ****PATCH / PUT****: altera ou atualiza um elemento existente; - ****DELETE****: remove ou destrói um elemento.

Exemplos práticos:

- `GET /rooms` : mapeia para a listagem de todos os quartos cadastrados;
- `POST /rooms` : cria um novo quarto na coleção;
- `GET /rooms/123` : busca e exibe os detalhes do quarto com ID 123;
- `DELETE /rooms/123` : solicita a remoção do quarto identificado por 123.

2.4 Conhecendo os componentes

O Ruby on Rails adota rigidamente o padrão **Model-View-Controller (MVC)**:

- **Modelos (Models)**: Possuem duas responsabilidades essenciais: representam os dados da sua aplicação persistidos no banco de dados e encapsulam as **regras de negócio**, cálculos e validações (como verificar se uma senha atende aos requisitos de segurança, ou se o usuário pode avaliar seu próprio quarto).
- **Controle (Controllers)**: A camada intermediária entre a Web e o seu sistema de regras. O controller recebe a requisição do navegador, extrai os parâmetros enviados, invoca as operações necessárias nos modelos e prepara as variáveis que serão entregues para a camada de apresentação.
- **Apresentação (Views)**: Onde se define como o usuário visualiza o resultado das operações. Podem ser templates HTML com Ruby embutido (ERB), fragmentos de Turbo Stream ou respostas JSON para APIs.

2.5 Os modelos

No Rails, a persistência e a lógica de banco de dados são coordenadas pelo componente chamado **ActiveRecord**. O ActiveRecord é uma implementação consagrada do padrão *Object-Relational*

Mapping (ORM). Ele mapeia tabelas para classes Ruby e linhas da tabela para instâncias dessas classes, convertendo tipos e chaves estrangeiras em associações orientadas a objetos.

Internamente, o ActiveRecord utiliza o **Arel**, um construtor de consultas relacionais que transforma chamadas de métodos Ruby encadeados em consultas SQL seguras e eficientes:

```
RUBY

# Busca todos os quartos:
Room.all

# SELECT "rooms".* FROM "rooms"

# Busca quartos com filtros específicos:
Room.where(location: [%x27;São Paulo', 'Rio de Janeiro'])

# SELECT "rooms".* FROM "rooms" WHERE "rooms"."location" IN ('São Paulo', 'Rio de Janeiro')
```

Além disso, o ActiveRecord integra o componente **ActiveModel**, que fornece toda a infraestrutura de validações de atributos, conversão de formatos, integração com formulários e suporte a internacionalização de mensagens de erro.

Os modelos não precisam obrigatoriamente herdar de `ActiveRecord::Base` (ou `ApplicationRecord`). Em sistemas profissionais, é muito comum e saudável criar classes em Ruby puro que utilizam `ActiveModel::Model` para separar regras complexas ou formulários que não possuem tabela própria no banco. Veremos isso na prática no Capítulo 9 com a classe `UserSession`.

2.6 Controle

A camada de controle é o intermédio entre os dados que vêm dos usuários do site e os Modelos. O controller gerencia sessões e cookies, de forma que o usuário não precise enviar suas credenciais em cada requisição individual.

Após obter ou gravar os dados na camada de modelos, é papel do controller decidir a resposta apropriada: renderizar uma view completa, retornar uma resposta de redirecionamento HTTP (302/303) ou responder a requisições com Turbo Streams.

No Rails, os componentes fundamentais dessa camada são o **ActionDispatch** (que cuida dos detalhes de baixo nível do protocolo HTTP, cookies, tipos MIME e parâmetros) e o **ActionController** (onde escrevemos nossos métodos de ação e aplicamos filtros como `before_action`).

2.7 Apresentação

A camada de apresentação prepara a resposta final para o usuário. No Rails, construímos telas combinando HTML com a linguagem de templates **ERB** (Embedded Ruby) fornecida pelo **ActionView**.

O Rails também possui um pipeline de ativos modernos: no Rails 8, o **Propshaft** gerencia a entrega de arquivos estáticos (CSS, imagens, fontes) com cache baseado em digest, enquanto o **Importmaps** gerencia as bibliotecas JavaScript sem etapas complexas de compilação.

2.8 Rotas

O arquivo `config/routes.rb` é a tabela mestra onde você informa ao Rails quais URLs a aplicação reconhece e qual controller e ação devem ser invocados em resposta a cada requisição.

O fluxo canônico do MVC é ilustrado a seguir:

CODE

```
[Navegador do Usuário]
  | (1) Requisição HTTP: GET /rooms/1/edit
  v
[Servidor Web]
  |
  v
[Roteador Rails] (Mapeia para RoomsController#edit com params[:id] = 1)
  |
  v
[RoomsController]
  |
  |→ (2) Invoca Room.find(1) no ActiveRecord [Model]
  |← Retorna a instância do quarto (@room)
  |
  |→ (3) Passa @room para a View [edit.html.erb dentro de application.html.erb]
  v
[ActionView]
  | (4) Gera HTML completo
  v
[Resposta HTTP ao Navegador]
```

2.9 Suporte

Envolvendo todos esses componentes está o **ActiveSupport**. Trata-se de uma rica coleção de extensões à biblioteca padrão do Ruby, provendo métodos utilitários que tornam o código muito mais expressivo:

RUBY

```
2.days.ago
# ⇒ Retorna a data exata de dois dias atrás com fuso horário

1.week.from_now
# ⇒ Retorna a data daqui a uma semana

"casa de praia".parameterize
# ⇒ "casa-de-praia" (perfeito para geração de slugs!)
```

2.10 Considerações finais

Não se preocupe em memorizar todos esses nomes e siglas agora. O mais importante é compreender o fluxo do MVC e a noção de recursos:

****Regra de ouro do Rails:**** Sempre que possível, modele suas funcionalidades como recursos RESTful. E ****nunca**** coloque regras importantes de negócio ou persistência complexa diretamente dentro dos Controllers — as regras de negócio pertencem aos Modelos!

Com essa base consolidada, é hora de colocar a mão na massa.

Capítulo 3: Primeiros passos com Rails

**Conhecer seus limites já é estar à frente deles.* — Georg Wilhelm Friedrich Hegel*

3.1 Instalação do Ruby e Rails

Para iniciar o desenvolvimento profissional com Rails moderno, recomendamos utilizar um gerenciador de versões como o `rbenv` ou `asdf` no macOS ou Linux. No ambiente atual, estamos utilizando:

- **Ruby:** versão 3.4+
- **Rails:** versão 8.x
- **SQLite3:** banco de dados relacional ágil e moderno, configurado nativamente com transações concorrentes e modo WAL.

3.2 Gerar o alicerce da aplicação

Para criar o projeto `colchonnet`, executamos no terminal:

BASH

```
$ rails new colchonnet --skip-jbuilder
```

O Rails cria imediatamente a estrutura padrão de pastas:

- `app/`: O coração do seu código-fonte. Contém subpastas para `models`, `controllers`, `views`, `helpers`, `mailers` e `javascript/controllers` (Stimulus);
- `config/`: Configurações de rotas (`routes.rb`), banco de dados (`database.yml`), inicializadores (`initializers/`) e internacionalização (`locales/`);
- `db/`: Armazena o schema do banco de dados e os arquivos de migrações (`db/migrate/`);
- `lib/`: Código e utilitários da sua aplicação que não pertencem diretamente ao ciclo web, além de tarefas Rake customizadas (`lib/tasks/`);
- `public/`: Arquivos estáticos entregues diretamente pelo servidor web (como páginas de erro `404.html`, `500.html` e favicon);
- `storage/`: Onde ficam salvos os uploads locais gerenciados pelo Active Storage e os bancos SQLite;

- `test/`: Toda a suíte de testes automatizados (unitários, de sistema e integração).

3.3 Os ambientes de execução

O Rails possui nativamente três ambientes de execução:

- **development**: O ambiente padrão em nossa máquina local. O reload de classes é ativado por padrão: quando você altera um arquivo `.rb` ou template e atualiza a página, a alteração é refletida imediatamente sem necessidade de reiniciar o servidor;
- **test**: Ambiente isolado utilizado na execução da suíte de testes (`bin/rails test`). O banco de dados de teste é zerado e preparado a cada bateria para garantir reprodutibilidade;
- **production**: O ambiente de alta performance onde sua aplicação roda para usuários reais. O cache de classes e a pré-compilação de assets ficam ativados para garantir tempo de resposta mínimo.

3.4 Os primeiros comandos

Para iniciar o servidor web do Rails:

BASH

```
$ bin/rails server
```

O Puma iniciará ouvindo na porta 3000. Ao abrir `http://localhost:3000/` no navegador, você verá a clássica página de boas-vindas do Rails.

Vamos agora gerar o nosso primeiro recurso: o quarto (`Room`), que terá um título (`title`), uma localização (`location`) e uma descrição longa (`description:text`). Faremos isso através do gerador de modelo do Rails:

BASH

```
$ bin/rails generate model Room title:string location:string description:text
```

Esse comando cria o modelo `app/models/room.rb` e a migração correspondente em `db/migrate/`.

Para aplicar a migração e criar a tabela no banco:

BASH

```
$ bin/rails db:migrate
```

3.5 Os arquivos gerados pelo scaffold

Uma migração é uma especificação em Ruby que descreve uma alteração incremental no esquema do banco de dados. O método `change` sabe como aplicar a alteração (criar tabela) e também como desfazê-la caso você execute um rollback:

BASH

```
$ bin/rails db:rollback
```

Ao inspecionar a migração de criação de quartos:

RUBY

```
class CreateRooms < ActiveRecord::Migration[8.1]
  def change
    create_table :rooms do |t|
      t.string :title
      t.string :location
      t.text :description

      t.timestamps
    end
  end
end
```

Observe a chamada `t.timestamps`. O Rails cria e gerencia automaticamente duas colunas vitais para qualquer registro: `created_at` (preenchido no momento da criação) e `updated_at` (atualizado a cada alteração do registro).

Capítulo 4: Implementação do modelo para o cadastro de usuários

**É bom perseguir um objetivo para uma jornada, mas no fim o que realmente importa é a jornada em si.* – Ursula Guin*

Com a base estruturada, vamos construir uma das partes mais fundamentais do Colcho.net: o cadastro e autenticação de usuários.

4.1 O usuário

Para o modelo de usuários, precisamos dos seguintes dados:

- `full_name`: Nome completo;
- `email`: Endereço de e-mail (chave de login);
- `password_digest`: A senha cifrada;
- `location`: Localização (cidade e estado);
- `bio`: Biografia do usuário;
- `confirmed_at`: Timestamp de verificação do e-mail;
- `confirmation_token`: Token aleatório de ativação.

Geramos o modelo e a migração:

BASH

```
$ bin/rails generate model user full_name:string email:string:uniq password_digest:string
```

NOTA

****Índices de Unicidade em Migrações**** Ao passar ``email:string:uniq``, o Rails adiciona automaticamente a diretiva: ``add_index :users, :email, unique: true`` Isso instrui o motor do banco de dados a rejeitar a nível de armazenamento qualquer tentativa de inserir e-mails duplicados.

4.2 Evite dados errados. Faça validações

O ActiveRecord disponibiliza uma rica DSL declarativa para validações. No modelo `User` (`app/models/user.rb`), precisamos assegurar que:

- 1. Nome, e-mail e localização sejam sempre preenchidos;
- 2. A bio possua conteúdo relevante (mínimo de 30 caracteres);
- 3. O e-mail tenha formato válido e não esteja em uso;
- 4. A senha seja confirmada sem erros de digitação.

RUBY

```
class User < ApplicationRecord
  EMAIL_REGEX = /\A[^@]+@([^\.\.]+\.)+[^@\.]+\z/

  has_secure_password

  validates :full_name, :email, :location, presence: true
  validates :bio, length: { minimum: 30 }, allow_blank: false
  validates :email, format: { with: EMAIL_REGEX }, uniqueness: true
end
```

Ao rodar `bin/rails console`, podemos verificar o comportamento das validações:

RUBY

```
user = User.new
user.valid? # => false
user.errors.messages
# => {:full_name=>["não pode ficar em branco"], :email=>["não pode ficar em branco", "não
```

O método `user.save` executa `valid?` internamente. Se algum atributo for inválido, o registro não é salvo no banco e retorna `false`.

Capítulo 5: Tratando as requisições Web

**Eu não desanimo, pois cada tentativa incorreta descartada é mais um passo à frente.* – Thomas A. Edison*

5.1 Roteie as requisições para o controle

Para conectar as requisições web ao modelo de usuário, declaramos no arquivo `config/routes.rb`:

```
Rails.application.routes.draw do
  resources :users
  root "home#index"
end
```

Essa única linha `resources :users` cria as sete rotas REST padrão (`index`, `new`, `create`, `show`, `edit`, `update`, `destroy`).

Ao acessar `http://localhost:3000/users/new` no navegador antes de criar o controller, o Rails exibirá um erro explicativo de roteamento: `uninitialized constant UsersController`.

Criamos então o controller em `app/controllers/users_controller.rb`:

```
class UsersController < ApplicationController
  def new
    @user = User.new
  end
end
```

NOTA

****Falsificação de Requisição (CSRF)**** O `ApplicationController` herda de `ActionController::Base` e inclui proteção CSRF nativa. Para cada formulário gerado pelo Rails com `form_for` ou `form_with`, um token de autenticidade único é embutido na página em uma tag oculta `<%= form_authenticity_token %>`

5.2 Integre o controle e a apresentação

Toda variável de instância definida na ação do controller (com o prefixo `@`, como `@user`) fica automaticamente disponível dentro do template daquela ação em `app/views/users/new.html.erb`.

Construímos o formulário com o construtor `form_for`:

ERB

```

<h1>Cadastro</h1>

<%= form_for @user do |f| %>
  <p>
    <%= f.label :full_name %><br>
    <%= f.text_field :full_name %>
  </p>
  <p>
    <%= f.label :location %><br>
    <%= f.text_field :location %>
  </p>
  <p>
    <%= f.label :email %><br>
    <%= f.text_field :email %>
  </p>
  <p>
    <%= f.label :password %><br>
    <%= f.password_field :password %>
  </p>
  <p>
    <%= f.label :password_confirmation %><br>
    <%= f.password_field :password_confirmation %>
  </p>
  <p>
    <%= f.label :bio %><br>
    <%= f.text_area :bio %>
  </p>
  <p>
    <%= f.submit %>
  </p>
<% end %>

```

5.3 Controle o mass-assignment com Strong Parameters

Quando o formulário é submetido, os dados chegam encapsulados no hash `params[:user]`. No Rails contemporâneo, passar diretamente `User.new(params[:user])` gera a exceção `ActiveModel::ForbiddenAttributesError`.

Isso ocorre porque um usuário malicioso poderia forjar atributos extras na requisição (por exemplo, `admin=1`). Para prevenir isso, usamos **Strong Parameters**:

```
private

def user_params
  params.require(:user).permit(
    :full_name,
    :email,
    :location,
    :password,
    :password_confirmation,
    :bio
  )
end
```

A ação `create` no `UserController` fica então:

```
def create
  @user = User.new(user_params)
  if @user.save
    redirect_to @user, notice: &#x27;Cadastro criado com sucesso!&#x27;
  else
    render :new, status: :unprocessable_entity
  end
end
```

IMPORTANTE

****redirect_to vs. render**** Se o registro foi salvo com sucesso, redirecionamos com ``redirect_to @user``: isso envia um status HTTP 302/303 instruindo o navegador a fazer uma nova requisição GET para a página de exibição. Se houver erros de validação, ****nunca**** redirecionamos: usamos ``render :new, status: :unprocessable_entity``. O ``render`` reutiliza a instância ``@user`` já preenchida e exibe a tela com todas as mensagens de erro populadas, sem que o usuário perca o que digitou.

5.4 Exibição do perfil do usuário

Na ação `show`, buscamos o usuário pelo ID vindo da URL (`params[:id]`):

RUBY

```
def show
  @user = User.find(params[:id])
end
```

E no template `app/views/users/show.html.erb`:

ERB

```
<h1>Perfil: <%= @user.full_name %></h1>
<ul>
  <li>Localização: <%= @user.location %></li>
  <li>Bio: <%= @user.bio %></li>
</ul>

<%= link_to 'Editar Perfil', edit_user_path(@user) %>
```

5.5 Permita a edição do perfil

A ação `edit` e `update` completam o ciclo de manutenção do perfil:

RUBY

```
def edit
  @user = User.find(params[:id])
end

def update
  @user = User.find(params[:id])
  if @user.update(user_params)
    redirect_to @user, notice: &#x27;Cadastro atualizado com sucesso!&#x27;
  else
    render :edit, status: :unprocessable_entity
  end
end
```

5.6 Reaproveite as apresentações com partials

Como o formulário de cadastro (`new`) e o de edição (`edit`) compartilham praticamente os mesmos campos, extraímos o formulário para uma **partial** nomeada com sublinhado no início: `app/views/users/_form.html.erb`.

Nos templates `new.html.erb` e `edit.html.erb`, chamamos a partial simplesmente com:

```
<%= render 'form' %>
```

ERB

O Rails busca automaticamente pelo arquivo `_form.html.erb` no diretório correspondente.

5.7 Mostre os erros no formulário

Quando campos do formulário contêm erros de validação, o Rails automaticamente envolve as tags em elementos com a classe CSS `.field_with_errors`.

Podemos verificar se há erros com `@user.errors.any?` e listar as mensagens detalhadas:

```
<% if @user.errors.any? %>
  <div id="error_explanation">
    <h2>Infelizmente não foi possível completar a ação:</h2>
    <ul>
      <% @user.errors.full_messages.each do |message| %>
        <li><%= message %></li>
      <% end %>
    </ul>
  </div>
<% end %>
```

ERB

5.8 Configure a ação raiz (root)

Definimos a página inicial do site em `config/routes.rb`:

```
root &#x27;home#index'
```

RUBY

Criamos o controller `app/controllers/home_controller.rb`:

```
RUBY

class HomeController < ApplicationController
  def index
    @rooms = Room.most_recent.limit(3)
  end
end
```

E a view `app/views/home/index.html.erb` com os links principais da aplicação.

Capítulo 6: Melhore o projeto

**Se cheguei até aqui, foi porque me apoiei no ombro de gigantes.* — Isaac Newton*

6.1 Lição obrigatória: sempre aplique criptografia para armazenar senhas

Armazenar senhas em texto puro é uma falha gravíssima de segurança. O Rails oferece integração padrão com o algoritmo **BCrypt** através da macro `has_secure_password`.

6.2 Como adicionar plugins e dependências ao projeto?

O gerenciamento de dependências no ecossistema Ruby é realizado pelo **Bundler**. Todas as bibliotecas de terceiros (chamadas *gems*) são declaradas no arquivo `Gemfile`:

```
gem 'bcrypt', '~> 3.1.7'
```

RUBY

Após declarar no `Gemfile`, executamos no terminal:

```
$ bundle install
```

BASH

O Bundler resolve as versões exatas e registra o grafo de dependências no arquivo imutável `Gemfile.lock`.

6.3 Usando `has_secure_password` no modelo

Ao declarar `has_secure_password` no modelo `User`:

- ↳5. Ele exige que exista uma coluna `password_digest` na tabela de usuários;
- ↳6. Ele cria automaticamente os métodos acessores virtuais `password` e `password_confirmation`;
- ↳7. Ele adiciona validações de presença da senha na criação e de confirmação de senha;
- ↳8. Ele adiciona o método `user.authenticate("senha")`, que retorna a própria instância de `User` se a senha for válida ou `false` caso contrário.

6.4 Migração da tabela users

Caso a tabela de usuários já existisse sem o campo `password_digest`, criaríamos uma migração incremental:

BASH

```
$ bin/rails generate migration add_password_digest_to_users password_digest:string
$ bin/rails db:migrate
```

6.5 Automatizando tarefas de manutenção com Rake

Tarefas de manutenção de banco de dados ou migração de dados legados são escritas como tarefas Rake em `lib/tasks/`:

RUBY

```
namespace :app do
  desc "Encripta senhas legadas no banco de dados"
  task migrar_senhas: :environment do
    User.find_each do |user|
      # Processamento em lote sem sobrecarregar a memória
    end
  end
end
```

💡 DICA

****Por que usar `find_each` em vez de `all`?**** Ao consultar milhares de registros, `User.all` tenta carregar todos os objetos na memória RAM de uma única vez, o que pode derrubar o servidor por falta de memória. O método `find_each` busca os registros em lotes de 1.000 por padrão, processa e libera a memória progressivamente.

6.6 Melhoria de templates e CSS

O CSS gerado por padrão pelo scaffold é básico. Podemos refinar visualmente o Colcho.net criando uma identidade acolhedora:

- Topo com tom verde suave (`#9ECE71`);
- Tipografia moderna para o logotipo com a fonte *Pacifico* do Google Fonts;

- Menu de navegação superior com botões de cantos arredondados;
- Caixas de formulário limpas e feedback de erro claro.

6.7 Trabalhe com layout e templates para melhorar sua apresentação

O layout principal `app/views/layouts/application.html.erb` envolve todas as páginas do site:

ERB

```
<!DOCTYPE html>
<html>
  <head>
    <title>ColchoNet</title>
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <link href="https://fonts.googleapis.com/css?family=Pacifico" rel="stylesheet" type="
    <%= csrf_meta_tags %>
    <%= csp_meta_tag %>
    <%= stylesheet_link_tag "application", "data-turbo-track": "reload" %>
    <%= javascript_importmap_tags %>
  </head>
  <body>
    <header>
      <div id="header-wrap">
        <h1><%= link_to "colcho.net", root_path %></h1>
        <!-- Navbars -->
      </div>
    </header>
    <div id="content">
      <% if notice.present? %><p id="notice"><%= notice %></p><% end %>
      <% if alert.present? %><p id="alert"><%= alert %></p><% end %>
      <%= yield %>
    </div>
  </body>
</html>
```

A diretiva `<%= yield %>` indica o ponto exato onde o conteúdo do template da ação específica será injetado.

6.8 O que é o Asset Pipeline contemporâneo (Propshaft e Importmaps)?

No Rails 4, o Asset Pipeline utilizava o motor Sprockets para compilar arquivos Sass/CoffeeScript, concatenar tudo em um único arquivo gigante e minificar.

No **Rails 8**, a web moderna evoluiu:

- 53. Os navegadores possuem suporte nativo a módulos JavaScript (ESM). Com o **Importmap-rails**, não precisamos mais de compiladores de bundling (como Webpack ou Vite) para a grande maioria das aplicações: o Rails mapeia pacotes diretamente para o navegador;
- 54. O **Propshaft** atua como um pipeline de ativos leve, focado em servir arquivos com digest (para invalidação de cache) sem complexidade desnecessária.

6.9 Criando os novos stylesheets

Criamos o arquivo `app/assets/stylesheets/application.css` estruturando os seletores de cabeçalho, caixas de conteúdo `#content`, estilos de botões, cartões de quartos e campos de formulário.

6.10 Feedback em erros de formulário

Em vez de exibir um bloco genérico no topo e deixar o usuário adivinhando qual campo falhou, é muito mais amigável exibir a mensagem de erro específica logo abaixo do campo com problemas.

6.11 Duplicação de lógica na apresentação nunca mais. Use os Helpers

Para evitar repetir blocos `<% if model.errors[:campo].any? %>` em cada input de formulário, criamos um método no `ApplicationHelper`:

RUBY

```
module ApplicationHelper
  def error_tag(model, attribute)
    return unless model&.errors&.key?(attribute)

    content_tag(
      :div,
      model.errors[attribute].first,
      class: "error_message"
    )
  end
end
```

Agora, em qualquer formulário, basta chamar:

ERB

```
<%= f.text_field :full_name %>
<%= error_tag @user, :full_name %>
```

Capítulo 7: Faça sua aplicação falar várias línguas

**Lembrar-se de que morreremos um dia é a melhor forma que conheço para evitar a armadilha de pensar que temos algo a perder.* — Steve Jobs*

7.1 O processo de internacionalização (I18n)

O sistema de internacionalização do Rails baseia-se em um dicionário de chave-valor organizado em árvore hierárquica em arquivos YAML (`config/locales/`).

Para configurar o português brasileiro como idioma padrão da aplicação:

RUBY

```
# config/application.rb
config.i18n.default_locale = :pt-BR
```

Criamos o arquivo `config/locales/pt-BR.yml`:

YAML

```
pt-BR:
  activerecord:
    models:
      user: "Usuário"
      room: "Quarto"
    attributes:
      user:
        full_name: "Nome completo"
        email: "Email"
        password: "Senha"
        password_confirmation: "Confirme sua senha"
        location: "Localização"
        bio: "Biografia"
```

Com essa estrutura, os helpers de formulário traduzem os rótulos `<%= f.label :full_name %>` automaticamente para "Nome completo", sem que você precise escrever strings fixas no HTML!

7.2 Traduza os templates

Nos templates, chamamos o helper `t()` (atalho para `I18n.t`):

ERB

```
<h1><%= t '.title' %></h1>
```

DICA

****Tradução com Escopo Relativo (Lazy Lookup)**** Quando você inicia a chave com um ponto (por exemplo, `t('title')`), o Rails deduz o escopo completo a partir do controller e da view atual. Se chamado em `app/views/users/new.html.erb`, o Rails buscará automaticamente a chave `pt-BR.users.new.title`.

Para tradução de datas e horários, usamos o helper `l()` (atalho para `I18n.l` ou `localize`), que formata datas de acordo com os padrões culturais do idioma configurado:

ERB

```
<%= l(room.created_at, format: :short) %>
```

7.3 Extra: alterar o idioma do site

A melhor forma de permitir que o usuário alterne o idioma sem quebrar caches de navegação é incluir o idioma na própria URL:

`/pt-BR/rooms` ou `/en/rooms`.

Em `config/routes.rb`:

RUBY

```
LOCALES = /en|pt\-BR/

scope "(:locale)", locale: LOCALES do
  resources :rooms
  resources :users
  # ...
end
```

Eno `ApplicationController`:

RUBY

```
before_action do
  I18n.locale = params[:locale] || I18n.default_locale
end

def default_url_options
  { locale: (I18n.locale = I18n.default_locale ? nil : I18n.locale) }
end
```

Capítulo 8: O cadastro do usuário e a confirmação da identidade

**Suor mais sacrifício é igual a sucesso.* — Charles Finley*

8.1 Entenda o ActionMailer e use o MailCatcher / Letter Opener

O **ActionMailer** é o módulo do Rails responsável pelo envio de e-mails. Ele funciona de forma análoga a um controller: possui ações que recebem parâmetros e renderizam templates de e-mail.

Para testar o envio em desenvolvimento sem disparar e-mails reais, ferramentas locais interceptam as mensagens via SMTP e as exibem em uma interface gráfica no navegador.

8.2 Templates de email, eu preciso deles?

É uma excelente prática enviar e-mails no formato **multipart** (oferecendo tanto uma versão formatada em HTML quanto uma versão legível em texto puro). O Rails faz isso automaticamente se você fornecer os dois templates:

- `app/views/signup_mailer/confirm_email.html.erb`
- `app/views/signup_mailer/confirm_email.text.erb`

⚠ ATENÇÃO

****Atenção aos Links nos Mailers**** Em templates de e-mail, ****nunca**** use helpers terminados em ``_path`` (como ``root_path``), pois eles geram caminhos relativos (``/``). Em e-mails, o leitor não está dentro do seu domínio, portanto você deve sempre usar helpers com o sufixo ``_url`` (como ``root_url`` ou ``confirmation_url``), que geram endereços absolutos incluindo o protocolo e o host.

8.3 Mais emails e a confirmação da conta de usuário

Para evitar contas com e-mails falsos, geramos um token aleatório na criação do usuário (`confirmation_token`). Enviamos o token no link de confirmação do e-mail:

```
http://localhost:3000/confirmation?token= ...
```

Quando o usuário acessa essa URL, registramos o momento no campo `confirmed_at` e limpamos o token.

8.4 Um pouco de callbacks para realizar tarefas pontuais

No modelo `User`, usamos o callback `before_create` para atribuir o token aleatório com segurança:

```
RUBY

before_create do |user|
  user.confirmation_token = SecureRandom.urlsafe_base64
end
```

E implementamos os métodos de ciclo de vida:

```
RUBY

def confirm!
  return if confirmed?

  self.confirmed_at = Time.current
  self.confirmation_token = ""
  save!
end

def confirmed?
  confirmed_at.present?
end
```

8.5 Roteamento com restrições e controllers singleton

A confirmação de conta não é uma coleção de itens: trata-se de uma ação pontual. No Rails, modelamos isso como um **recurso singleton**:

```
RUBY

# config/routes.rb
resource :confirmation, only: [:show]
```

E criamos o `ConfirmationsController` (`app/controllers/confirmations_controller.rb`):

RUBY

```
class ConfirmationsController < ApplicationController
  def show
    user = User.find_by(confirmation_token: params[:token])
    if user.present?
      user.confirm!
      user_session.store(user) # Autentica o usuário automaticamente
      redirect_to user, notice: t(8#x27;confirmations.success')
    else
      redirect_to root_path, alert: t(8#x27;confirmations.invalid_token')
    end
  end
end
```

8.6 Em resumo

Com isso, o ciclo de cadastro e validação de identidade está completo:

57. O usuário preenche o formulário;
 58. Os dados são validados e a senha é criptografada com BCrypt;
 59. O callback gera um token criptograficamente seguro;
 50. O ActionMailer envia o e-mail com link exclusivo;
 51. O usuário clica no link e o controller ativa a conta.
-

Capítulo 9: Login do usuário

**Foco é uma questão de decidir as coisas que você não irá fazer.* – John Carmack, cofundador da id Software*

9.1 Trabalhe com a sessão

O protocolo HTTP é *stateless* (não guarda estado entre uma requisição e outra). Para manter um usuário conectado após efetuar o login, precisamos persistir um identificador na sessão do navegador.

O Rails gerencia isso através do cookie de sessão. O conteúdo é assinado digitalmente ou encriptado pelo servidor utilizando a chave secreta da aplicação (`secret_key_base`), impedindo que o usuário adultere o valor do seu próprio `user_id`.

9.2 Controles e rotas para o novo recurso

Modelamos a sessão do usuário como o recurso `user_sessions`:

RUBY

```
# config/routes.rb
resource :user_sessions, only: [:create, :new, :destroy]
```

9.3 Sessões e cookies

Para lidar com o formulário de login de forma elegante e desacoplada, criamos uma classe Ruby com `ActiveModel::Model`:

RUBY

```
class UserSession
  include ActiveRecord::Model

  attr_accessor :email, :password

  validates :email, :password, presence: true

  def initialize(session, attributes = {})
    @session = session
    @email = attributes[:email] if attributes
    @password = attributes[:password] if attributes
  end

  def authenticate!
    user = User.authenticate(@email, @password)
    if user.present?
      store(user)
      true
    else
      errors.add(:base, :invalid_login)
      false
    end
  end

  def store(user)
    @session[:user_id] = user.id
  end

  def destroy
    @session[:user_id] = nil
  end
end
```

9.4 Consultas no banco de dados

O ActiveRecord fornece métodos de busca que correspondem a cláusulas SQL:

- `.where(condicoes)`
- `.order(campo)`
- `.limit(qtd)`
- `.select(campos)`

🚫 CUIDADO

****O que é SQL Injection e como se proteger?**** Se você interpolar dados digitados pelo usuário diretamente em uma string SQL (`'User.where("email = '#{params[:email]}')'`), um invasor pode passar comandos maliciosos e obter controle total do seu banco de dados. O ActiveRecord previne isso automaticamente quando você passa argumentos parametrizados com hash ou array: `'User.where(email: params[:email])'` ou `'User.where("email = ?", params[:email])'`. Nesses casos, o ActiveRecord sanitiza e escapa os valores com rigor.

9.5 Escopo de usuário confirmado

Para impedir que usuários que ainda não clicaram no link do e-mail consigam fazer login, definimos um escopo nomeado no model `User`:

RUBY

```
scope :confirmed, → { where.not(confirmed_at: nil) }

def self.authenticate(email, password)
  user = confirmed.find_by(email: email)
  user.try(:authenticate, password)
end
```

Capítulo 10: Controle de acesso

**Formação em Ciência da Computação consegue tornar qualquer pessoa um excelente programador tanto quanto estudar pincéis e pigmentos torna qualquer pessoa um excelente pintor.* – Eric Raymond*

10.1 Helpers de sessão

No `ApplicationController`, centralizamos a lógica de autenticação disponível para todos os controllers e views:

```
RUBY

delegate :current_user, :user_signed_in?, to: :user_session
helper_method :current_user, :user_signed_in?

def user_session
  UserSession.new(session)
end

def require_authentication
  unless user_signed_in?
    redirect_to new_user_sessions_path, alert: t("flash.alert.needs_login")
  end
end

def require_no_authentication
  if user_signed_in?
    redirect_to root_path, notice: t("flash.notice.already_logged_in")
  end
end
```

E no layout, exibimos o menu contextual:

- Para visitantes: Quartos, Cadastro e Login;
- Para usuários logados: Cadastre seu quarto!, Quartos, Meu perfil e Logout.

10.2 Não permita edição do perfil alheio

Para evitar que um usuário mal-intencionado edite o perfil de outro usuário simplesmente mudando o ID na URL (`/users/42/edit`), adicionamos um filtro:

RUBY

```
before_action :can_change, only: [:edit, :update]

def can_change
  unless user_signed_in? && current_user == user
    redirect_to user_path(params[:id])
  end
end

def user
  @user ||= User.find(params[:id])
end
```

10.3 Relacionando seus objetos

Para modelar que um usuário é o proprietário de seus quartos, usamos as associações fundamentais do ActiveRecord:

- 1 para 1: `has_one` / `belongs_to`
- 1 para Muitos: `has_many` / `belongs_to`
- Muitos para Muitos: `has_many :through`

10.4 Relacione quartos a usuários

No modelo `User`:

RUBY

```
has_many :rooms, dependent: :destroy
```

No modelo `Room`:

RUBY

```
belongs_to :user
```

10.5 Limite o acesso usando relacionamentos

Para garantir segurança total no gerenciamento de quartos, vinculamos a busca de edição e exclusão ao escopo do usuário autenticado:

RUBY

```
# Em RoomsController:
def edit
  @room = current_user.rooms.find(params[:id])
end

def update
  @room = current_user.rooms.find(params[:id])
  if @room.update(room_params)
    redirect_to @room, notice: t("flash.notice.room_updated")
  else
    render :edit, status: :unprocessable_entity
  end
end

def destroy
  @room = current_user.rooms.find(params[:id])
  @room.destroy

  redirect_to rooms_url, notice: t("flash.notice.room_destroyed"), status: :see_other
end
```

Se o quarto com aquele ID não pertencer ao usuário logado, `current_user.rooms.find` dispara automaticamente `ActiveRecord::RecordNotFound`, que o Rails trata como página não encontrada (404), sem vaziar dados.

10.6 Exibição e listagem de quartos

Na listagem de quartos, exibimos os botões de "Editar" e "Remover" apenas se o usuário atual for o criador daquele anúncio:

RUBY

```
# app/helpers/rooms_helper.rb
module RoomsHelper
  def belongs_to_user(room)
    user_signed_in? && room.user == current_user
  end
end
```

Capítulo 11: Avaliação de quartos, relacionamentos muitos para muitos e organização do código

**Ruby foi construído para tornar os programadores mais felizes.* – Yukihiro "Matz" Matsumoto*

11.1 Relacionamentos muitos-para-muitos

Um quarto recebe avaliações de vários usuários, e cada usuário pode avaliar vários quartos. Criamos o modelo `Review` com as chaves estrangeiras `user_id` e `room_id`:

```
RUBY

class CreateReviews < ActiveRecord::Migration[8.1]
  def change
    create_table :reviews do |t|
      t.references :user, null: false, foreign_key: true
      t.references :room, null: false, foreign_key: true
      t.integer :points
      t.index [:user_id, :room_id], unique: true

      t.timestamps
    end
  end
end
```

O índice composto único garante no banco que um usuário nunca vote mais de uma vez no mesmo quarto.

11.2 Removendo objetos sem deixar rastros

Para evitar registros órfãos quando um quarto ou usuário for deletado, especificamos:

```
RUBY

has_many :reviews, dependent: :destroy
```

11.3 Criando avaliações reativas com Hotwire / Turbo Streams

Na versão original do livro, utilizava-se `remote: true`, jQuery e eventos `ajax:success`. No Rails 8, a abordagem moderna é utilizar **Turbo Streams** e **Stimulus**:

Aninhamos as rotas de reviews em `config/routes.rb`:

```
RUBY

resources :rooms do
  resources :reviews, only: [:create, :update], module: :rooms
end
```

E no controller `Rooms::ReviewsController` (`app/controllers/rooms/reviews_controller.rb`):

RUBY

```

module Rooms
  class ReviewsController < ApplicationController
    before_action :require_authentication

    def create
      review = room.reviews.find_or_initialize_by(user_id: current_user.id)
      review.update!(review_params)

      respond_to do |format|
        format.html { redirect_to room_path(room), notice: t("flash.notice.review_created") }
        format.turbo_stream
      end
    end

    def update
      create
    end

    private

    def room
      @room ||= Room.find(params[:room_id])
    end

    def review_params
      params.require(:review).permit(:points)
    end
  end
end

```

11.4 Diga adeus a regras complexas de apresentação: use presenters

O padrão **Presenter** encapsula regras de formatação e exibição que não pertencem nem ao modelo de banco nem ao controller. Criamos o `RoomPresenter` (`app/presenters/room_presenter.rb`):

RUBY

```
class RoomPresenter
  delegate :user, :created_at, :description, :location, :title,
           :to_param, :to_key, :persisted?, :id, to: :@room

  def self.model_name
    Room.model_name
  end

  def initialize(room, context, show_form = true)
    @context = context
    @room = room
    @show_form = show_form
  end

  def can_review?
    @context.user_signed_in?
  end

  def show_form?
    @show_form
  end

  def stars
    @room.reviews.stars
  end

  def total_reviews
    @room.reviews.size
  end

  def to_partial_path
    "room"
  end
end
```

11.5 Da era jQuery ao Hotwire: requisições assíncronas modernas

A evolução do Rails substituiu scripts manuais por controllers declarativos do Stimulus. Criamos

```
app/javascript/controllers/review_stars_controller.js :
```

JAVASCRIPT

```

import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["form", "radio"]

  connect() {
    this.updateActiveStars()
  }

  hover(event) {
    const point = parseInt(event.currentTarget.dataset.point, 10)
    this.highlightUpTo(point)
  }

  resetHover() {
    this.updateActiveStars()
  }

  submit() {
    this.updateActiveStars()
    this.formTarget.requestSubmit()
  }

  updateActiveStars() {
    const checkedRadio = this.radioTargets.find(radio => radio.checked)
    const point = checkedRadio ? parseInt(checkedRadio.value, 10) : 0
    this.highlightUpTo(point)
  }

  highlightUpTo(point) {
    const labels = this.element.querySelectorAll(".star-label")
    labels.forEach(label => {
      const p = parseInt(label.dataset.point, 10)
      if (p ≤ point) {
        label.classList.add("toggled")
      } else {
        label.classList.remove("toggled")
      }
    })
  }
}

```

```

    }
  })
}
}

```

11.6 Média de avaliações usando agregações

No modelo `Review`:

```

def self.stars
  (average(:points) || 0).round
end

```

RUBY

Essa função roda via SQL nativo (`AVG("reviews"."points")`), tirando proveito dos índices sem instanciar centenas de objetos na memória do Ruby.

11.7 Aplicações modernas usam tipografia e ícones modernos

Substituímos sprites antigos por caracteres tipográficos escaláveis ou SVGs limpos, mantendo o visual de estrelas preenchidas (★) e vazias (☆) com estilo dourado e texto legível.

11.8 Eu vejo estrelas: usando CSS e Stimulus para melhorar as avaliações

No template de formulário de avaliação:

ERB

```

<div data-controller="review-stars">
  <% if room.can_review? %>
    <%= form_for room.review_route, html: { data: { review_stars_target: "form" }, class:
    <span class="stars-picker">
      <% room.review_points.each do |point| %>
        <%= f.radio_button :points, point, id: "points_#{room.id}_#{point}", data: { re
        <label for="<%= "points_#{room.id}_#{point}" %>" data-point="<%= point %>" data
        <% end %>
      </span>
      <%= f.submit t('.submit', default: 'Avaliar'), class: "review-submit-btn" %>
    <% end %>
  <% else %>
    <span class="login_required">
      <%= t('.login_to_review') %>
    </span>
  <% end %>
</div>

```

Ao clicar em qualquer estrela, o evento `change` dispara `requestSubmit()` via Stimulus, o Rails processa a avaliação e o template `create.turbo_stream.erb` substitui a seção da tela em tempo real!

Capítulo 12: Polindo o Colcho.net

**Não importa se você vai devagar, o que importa é você nunca parar.* — Confúcio*

12.1 Faça buscas textuais apenas com o Rails

No modelo `Room`:

```
RUBY

def self.search(query)
  if query.present?
    where("location LIKE :query OR title LIKE :query OR description LIKE :query", query:
  else
    all
  end
end
end
```

E no template, utilizamos o helper `highlight` do Rails para destacar visualmente a palavra buscada:

```
ERB

<%= link_to highlight(room.title, @search_query), room.route %>
```

12.2 URLs mais amigáveis através de slugs

Em vez de expor URLs genéricas com números como `/rooms/4`, o Rails permite criar slugs amigáveis como `/rooms/4-colchonete-na-sala` de forma nativa e sem precisar de dependências externas, bastando sobrescrever o método `#to_param` no modelo `Room`:

RUBY

```
def to_param
  return nil unless persisted?
  "#{id}-#{title.to_s.parameterize}"
end
```

Quando o Rails busca o quarto através de `Room.find(params[:id])`, o método converte automaticamente a string inicial para o número inteiro do ID.

12.3 Paginação de dados de forma descomplicada

Para paginar listagens de muitos quartos sem sobrecarregar o banco, utilizamos a gem `kaminari` integrada com o `RoomCollectionPresenter`:

RUBY

```
# Em RoomsController:
def index
  @search_query = params[:q]
  rooms = Room.search(@search_query).page(params[:page]).per(10)
  @rooms = RoomCollectionPresenter.new(rooms.most_recent, self)
end
```

E na view:

ERB

```
<%= paginate @rooms %>
```

12.4 Upload de fotos de forma simples com Active Storage

No Rails contemporâneo, não precisamos de gems legadas como CarrierWave ou RMagick. O **Active Storage** é integrado e nativo:

No modelo `Room`:

RUBY

```
has_one_attached :picture
```

No formulário `_form.html.erb`:

ERB

```
<%= f.label :picture %>
<%= f.file_field :picture, accept: "image/png,image/jpeg,image/gif,image/webp" %>
```

E no `RoomPresenter`, geramos a URL do thumbnail usando variantes com redimensionamento proporcional via `image_processing`:

RUBY

```
def thumb_url
  if @room.picture.attached?
    Rails.application.routes.url_helpers.rails_representation_url(
      @room.picture.variant(resize_to_limit: [100, 100]),
      only_path: true
    )
  end
end
```

12.5 Coloque a aplicação no ar

Para publicar a aplicação em produção no ecossistema moderno:

- O Rails 8 vem configurado com **Kamal** (`config/deploy.yml`) para deploy containerizado com Docker em qualquer VPS (como DigitalOcean ou AWS);
- Plataformas PaaS modernas (como Render, Fly.io ou Heroku) aceitam a aplicação com configuração simples de banco de dados PostgreSQL e chaves de ambiente.

Capítulo 13: Próximos passos

**Agora, isso não é o fim. Nem sequer é o início do fim. Mas é, talvez, o fim do começo.* — Sir Winston Churchill*

Parabéns! Você construiu o **Colcho.net** do zero até um sistema web completo e moderno, compreendendo as convenções, decisões de design, padrões de arquitetura e mecanismos de segurança do ecossistema Rails.

Onde aprofundar seus estudos:

- **RailsGuides:** Os guias oficiais em guides.rubyonrails.org são a documentação de referência mais completa sobre cada subsistema do framework;
- **Hotwire Handbook:** Leia a documentação do **Turbo** e **Stimulus** para criar interfaces ainda mais fluidas;
- **Test-Driven Development (TDD):** Pratique a escrita de testes com a suíte padrão Minitest ou RSpec;
- **Open Source:** Explore o código de gems populares no GitHub para aprender como desenvolvedores experientes estruturam seus pacotes em Ruby.

Bons códigos e que suas aplicações estejam sempre nos trilhos!